

Hands-On Activity 5 | Chart Type

```
# This is formatted as code
```

Name : Carl Andrei D. Torres

Course Code and Title : CPE 031 - Visualizations and Data Analysis

Date Submitted : August 22, 2026

Instructor : Engr. Jimlord Quejado

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

1. Objectives:

This activity aims to develop the students' ability to visualize and create data graphs.

2. Intended Learning Outcomes (ILOs): By the end of this module the students should be able to demonstrate:

1. Analyze data charts in various use cases.
2. Create effective data visualizations using charts.

3. Discussions:

Selecting the right chart type is important because it determines how clearly the data can be understood. Different charts serve different purposes—bar and column charts are best for comparing categories, line charts highlight trends over time, pie charts show proportions, and scatter or bubble charts reveal relationships between variables. Choosing the correct chart depends on the nature of the data and the problem you want to analyze. A well-selected chart makes patterns more visible, reduces confusion, and helps in drawing accurate insights.

4. Procedures:

Step 1 : Defining the problem using dataset

The data on road accident causes highlights a significant problem in road safety. By looking at the distribution, we can see that some causes occur far more often than others, showing clear patterns in why accidents happen. For example, if one category such as speeding or human error makes up the largest portion, it means that many accidents could be prevented by addressing that specific issue. This suggests that the problem is not just random but is linked to certain behaviors or conditions, and solving it will require targeted actions like stricter enforcement of traffic rules, better road designs, or awareness campaigns for drivers.

```
1 #Install pandas
2 !pip install pandas
```

Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
 Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.1.3)
 Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas) (2026.3)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import numpy as np
```

1. Create a local folder titled "CPE 031_SURNAME" that easily to locate.
2. Download the RoadAccidents.csv
3. Save it to the folder created.

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
1 df = pd.read_csv("/content/drive/MyDrive/CPE-031_Torres/RoadAccidents.csv")
```

1 df

7476 to 7500 of 9550 entries

Filter

?

index	Million Plus Cities	Cause category	Cause Subcategory	Outcome of Incident	Count
7475	Kollam	Impacting Vehicle/Object	Auto Rickshaws	Total number of Accidents	128.0
7476	Kollam	Impacting Vehicle/Object	Bicycles	Previously Injured	0.0
7477	Kollam	Impacting Vehicle/Object	Bicycles	Persons Killed	0.0
7478	Kollam	Impacting Vehicle/Object	Bicycles	Minor Injury	0.0
7479	Kollam	Impacting Vehicle/Object	Bicycles	Total number of Accidents	0.0
7480	Kollam	Impacting Vehicle/Object	Buses	Previously Injured	44.0
7481	Kollam	Impacting Vehicle/Object	Buses	Persons Killed	9.0
7482	Kollam	Impacting Vehicle/Object	Buses	Minor Injury	8.0
7483	Kollam	Impacting Vehicle/Object	Buses	Total number of Accidents	57.0
7484	Kollam	Impacting Vehicle/Object	Cars, Taxis, Vans and LMV	Previously Injured	424.0
7485	Kollam	Impacting Vehicle/Object	Cars, Taxis, Vans and LMV	Persons Killed	60.0
7486	Kollam	Impacting Vehicle/Object	Cars, Taxis, Vans and LMV	Minor Injury	67.0
7487	Kollam	Impacting Vehicle/Object	Cars, Taxis, Vans and LMV	Total number of Accidents	474.0
7488	Kollam	Impacting Vehicle/Object	Other Non	Previously Injured	0.0
7489	Kollam	Impacting Vehicle/Object	Other Non	Persons Killed	0.0
7490	Kollam	Impacting Vehicle/Object	Other Non	Minor Injury	0.0
7491	Kollam	Impacting Vehicle/Object	Other Non	Total number of Accidents	0.0
7492	Kollam	Impacting Vehicle/Object	Others	Previously Injured	17.0
7493	Kollam	Impacting Vehicle/Object	Others	Persons Killed	3.0
7494	Kollam	Impacting Vehicle/Object	Others	Minor Injury	0.0
7495	Kollam	Impacting Vehicle/Object	Others	Total number of Accidents	20.0
7496	Kollam	Impacting Vehicle/Object	Pedestrian	Previously Injured	0.0
7497	Kollam	Impacting Vehicle/Object	Pedestrian	Persons Killed	0.0
7498	Kollam	Impacting Vehicle/Object	Pedestrian	Minor Injury	0.0
7499	Kollam	Impacting Vehicle/Object	Pedestrian	Total number of Accidents	0.0

Show

25

 per page

1100200290299

300

301310380382

Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Next steps:

New interactive sheet

Step 2 : Deciding which data parameter is relevant according to the problem

In the case of road accidents, not all available data may directly explain the causes. For example, parameters like the cause category are very relevant because they show why accidents happen, while other details such as vehicle color may not contribute much to solving the problem. Choosing the right parameter ensures that the analysis focuses on meaningful information, avoids unnecessary complexity, and leads to more accurate conclusions and solutions.

Step 3 : Selecting a Chart Type

Choosing the correct chart is important because it determines how well the data can be understood and interpreted. The right chart makes patterns, comparisons, or proportions easier to see, while the wrong chart can confuse the audience or even hide the real meaning of the data. For example, a pie chart is best when showing proportions of a whole, such as the share of accident causes, while a bar chart is better for comparing exact numbers across categories. By matching the chart type to the purpose of the analysis, the data becomes clearer, more accurate, and more useful for decision-making.

Pie Chart

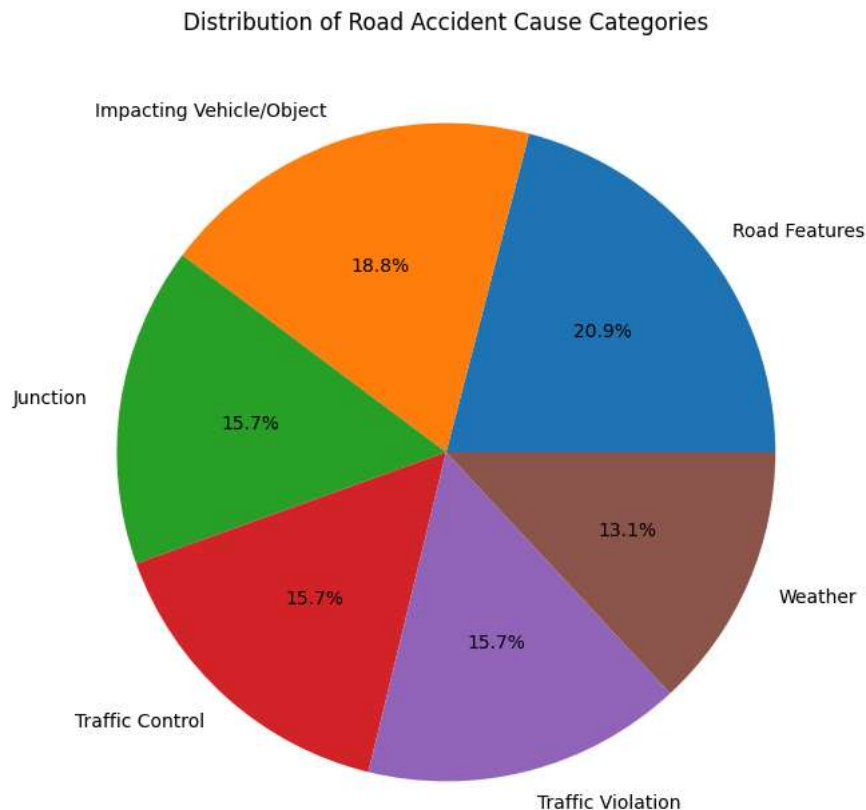
The pie chart is effective for this type of visualization because it clearly shows how road accident causes are divided into different categories. By using percentages, it is easy to compare the size of each category and quickly identify which cause contributes the most

and which ones are less common. This makes the data more understandable at a glance, especially when the goal is to highlight the proportion of each cause rather than the exact numbers.

```

1 # Count values in 'Cause category'
2 cause_counts = df['Cause category'].value_counts()
3
4 # Create a pie chart
5 cause_counts.plot.pie(figsize=(8, 8), autopct='%1.1f%%')
6
7 # Add title
8 plt.title('Distribution of Road Accident Cause Categories')
9
10 # Remove y-axis label
11 plt.ylabel('')
12
13 # Show chart
14 plt.show()

```



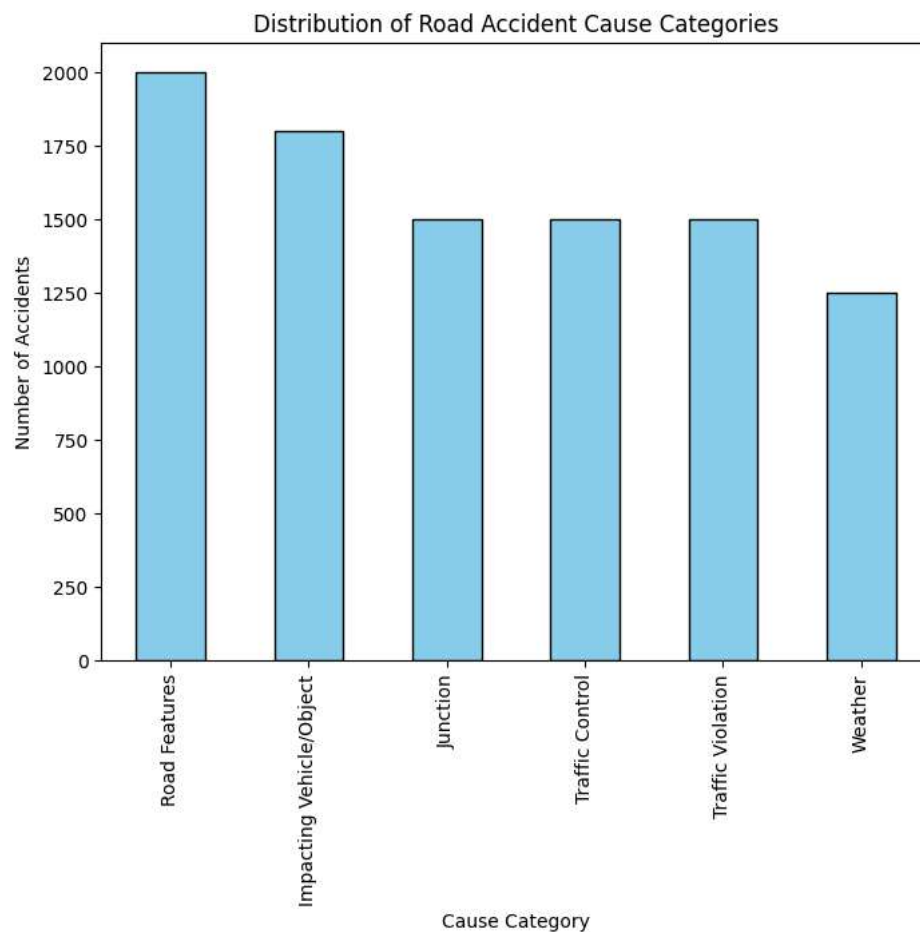
Bar Chart

The bar chart is effective for this type of visualization because it shows the exact number of accidents for each cause category. Unlike the pie chart, it makes it easier to compare the size of each category side by side, especially when there are many categories. The height of the bars gives a clear picture of which causes are more frequent and which are less common, making the data straightforward to interpret.

```

1 # Count values in the 'Cause category' column
2 cause_counts = df['Cause category'].value_counts()
3
4 # Plot a bar chart
5 cause_counts.plot.bar(figsize=(8, 6), color='skyblue', edgecolor='black')
6
7 # Add labels and title
8 plt.title('Distribution of Road Accident Cause Categories')
9 plt.xlabel('Cause Category')
10 plt.ylabel('Number of Accidents')
11
12 # Show chart
13 plt.show()

```

**Task 1.**

Using the dataset visualize the cause subcategory using the pie chart and bar chart.

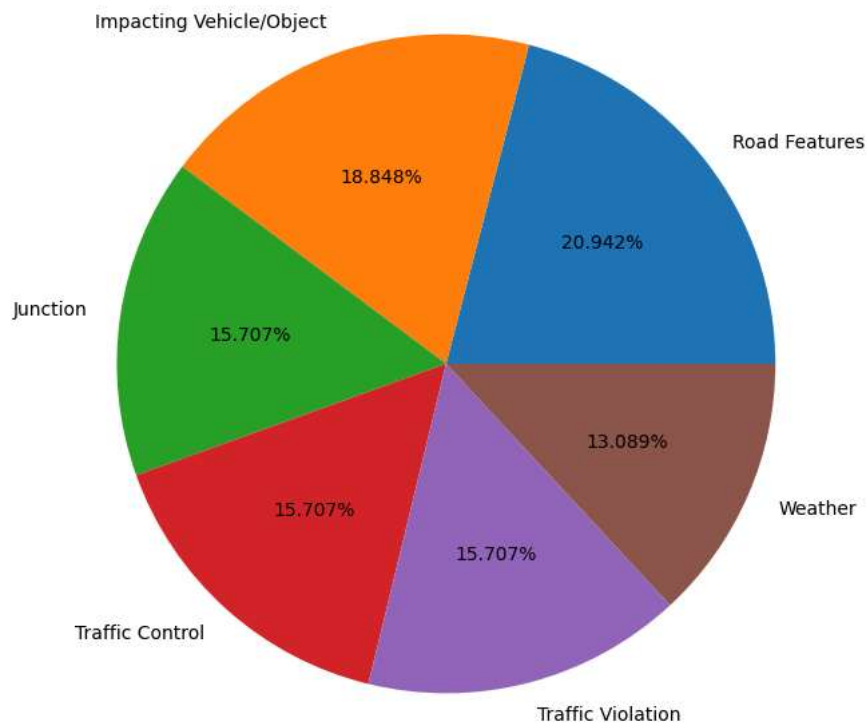
PIE

```

1 # Count values in 'Cause category'
2 cause_counts = df['Cause category'].value_counts()
3
4 # Create a pie chart
5 cause_counts.plot.pie(figsize=(8, 8), autopct='%1.3f%%')
6
7 # Add title
8 plt.title('Distribution of Road Accident Cause Categories')
9
10 # Remove y-axis label
11 plt.ylabel('')
12
13 # Show chart
14 plt.show()

```

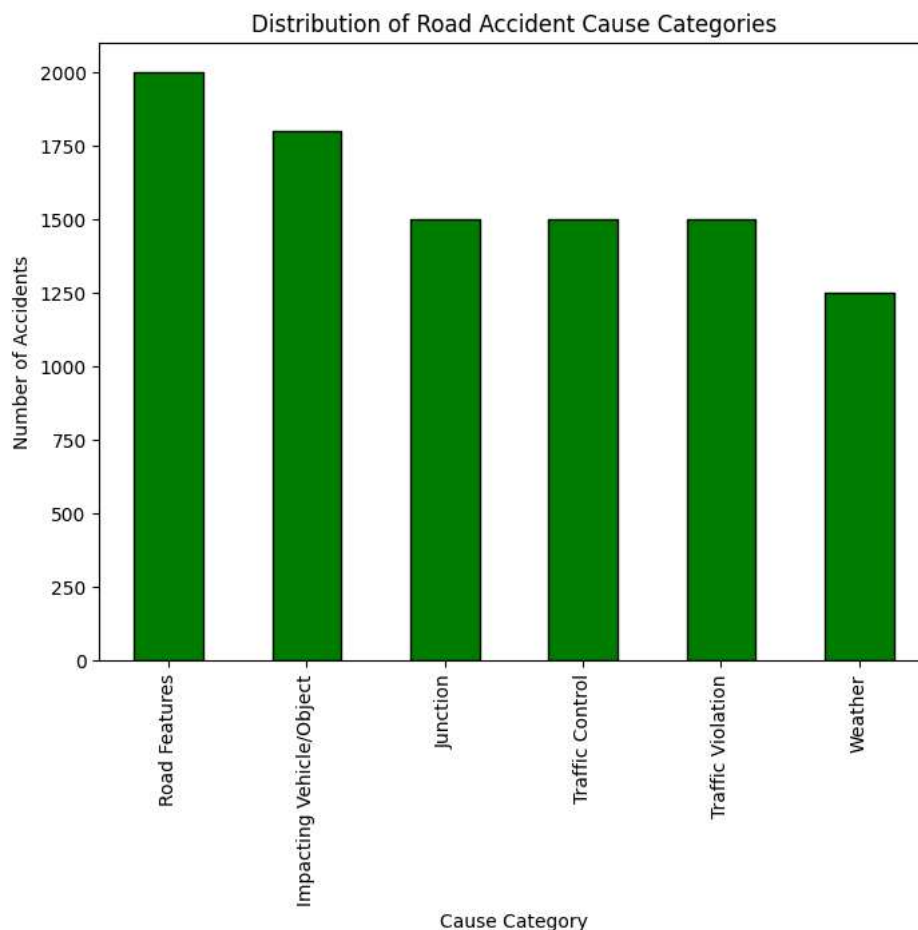
Distribution of Road Accident Cause Categories

**BAR**

```

1 # Count values in the 'Cause category' column
2 cause_counts = df['Cause category'].value_counts()
3
4 # Plot a bar chart
5 cause_counts.plot.bar(figsize=(8, 6), color='green', edgecolor='black')
6
7 # Add labels and title
8 plt.title('Distribution of Road Accident Cause Categories')
9 plt.xlabel('Cause Category')
10 plt.ylabel('Number of Accidents')
11
12 # Show chart
13 plt.show()

```



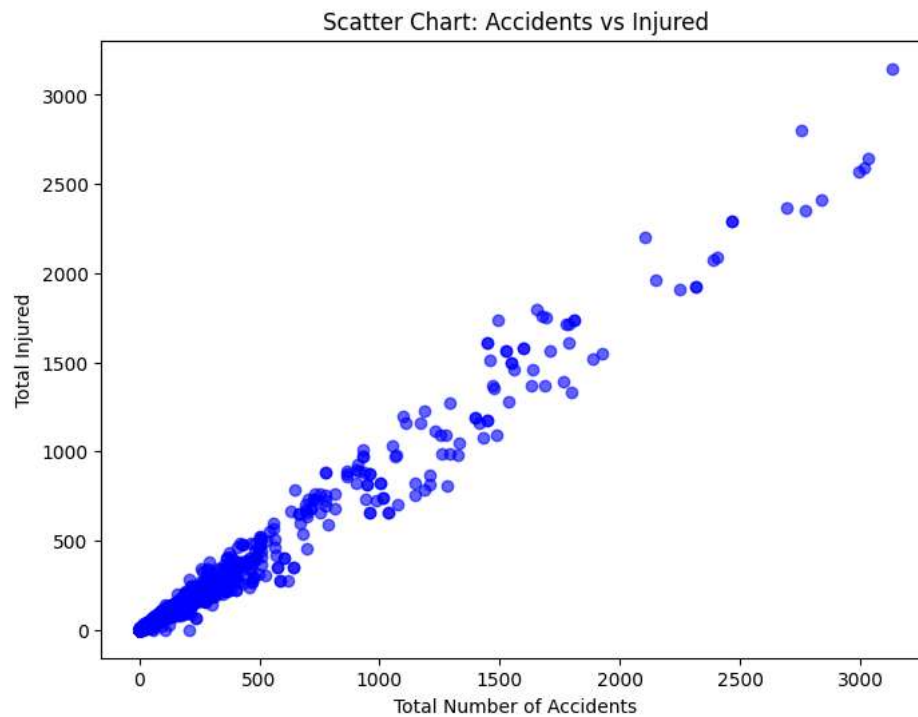
Scatter Chart

A scatter chart is best when you want to show the relationship between two numerical parameters. In your data, the "Count" column is numerical, so it should be one axis. The other axis can be a numerical transformation, such as grouping by "Cause category" or "Outcome of Incident" (converted to counts). For example, you can compare "Total number of Accidents" vs. "Total Injured" across different cities. This helps show whether more accidents always lead to more injuries.

```

1 # Filter dataset to only include rows with 'Total number of Accidents' and 'Total Injured'
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3 injured = df[df['Outcome of Incident'] == 'Total Injured']
4
5 # Merge on city and cause to align values
6 scatter_data = accidents.merge(
7     injured,
8     on=['Million Plus Cities', 'Cause category', 'Cause Subcategory'],
9     suffixes=('_Accidents', '_Injured')
10 )
11
12 # Plot scatter chart
13 plt.figure(figsize=(8, 6))
14 plt.scatter(
15     scatter_data['Count_Accidents'],
16     scatter_data['Count_Injured'],
17     alpha=0.6,
18     color='blue'
19 )
20
21 # Add labels and title
22 plt.title('Scatter Chart: Accidents vs Injured')
23 plt.xlabel('Total Number of Accidents')
24 plt.ylabel('Total Injured')
25
26 # Show chart
27 plt.show()

```



The scatter chart shows how the total number of accidents relates to the total number of injured. If the points go upward as accidents increase, it means more accidents usually lead to more injuries. Points that stand out show unusual cases where the number of injuries does not match the number of accidents.

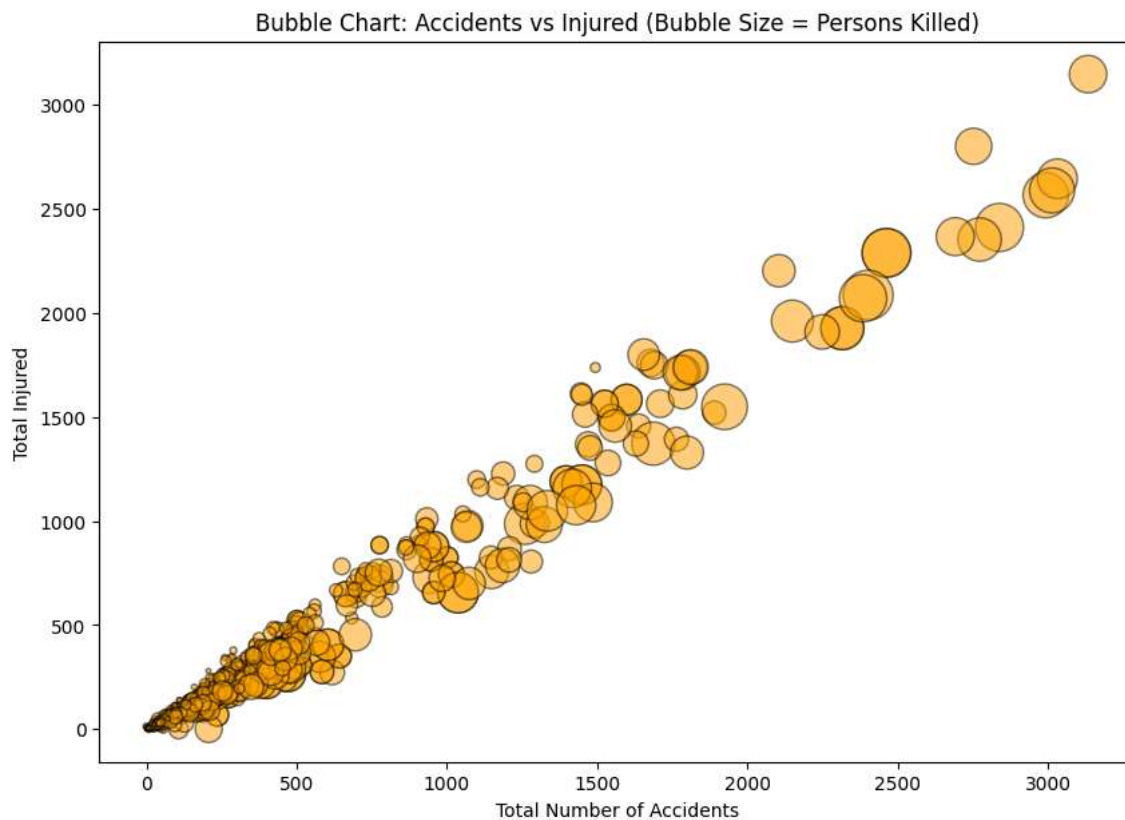
Bubble Chart

A bubble chart adds a third dimension by using bubble size. Here, the x-axis and y-axis can still be numerical values like “Total number of Accidents” and “Total Injured”, while the bubble size can represent “Persons Killed” or “Minor Injury” counts. You can also use “Million Plus Cities” or “Cause category” as grouping labels to distinguish bubbles by color.

```

1 # Filter dataset for needed outcomes
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3 injured = df[df['Outcome of Incident'] == 'Total Injured']
4 killed = df[df['Outcome of Incident'] == 'Persons Killed']
5
6 # Merge data on city and cause to align values
7 bubble_data = accidents.merge(
8     injured,
9     on=['Million Plus Cities', 'Cause category', 'Cause Subcategory'],
10    suffixes=('_Accidents', '_Injured')
11 ).merge(
12     killed,
13     on=['Million Plus Cities', 'Cause category', 'Cause Subcategory']
14 )
15
16 # Plot bubble chart
17 plt.figure(figsize=(10, 7))
18 plt.scatter(
19     bubble_data['Count_Accidents'],
20     bubble_data['Count_Injured'],
21     s=bubble_data['Count'], # bubble size = persons killed
22     alpha=0.5,
23     c='orange',
24     edgecolors='black'
25 )
26
27 # Add labels and title
28 plt.title('Bubble Chart: Accidents vs Injured (Bubble Size = Persons Killed)')
29 plt.xlabel('Total Number of Accidents')
30 plt.ylabel('Total Injured')
31
32 # Show chart
33 plt.show()

```

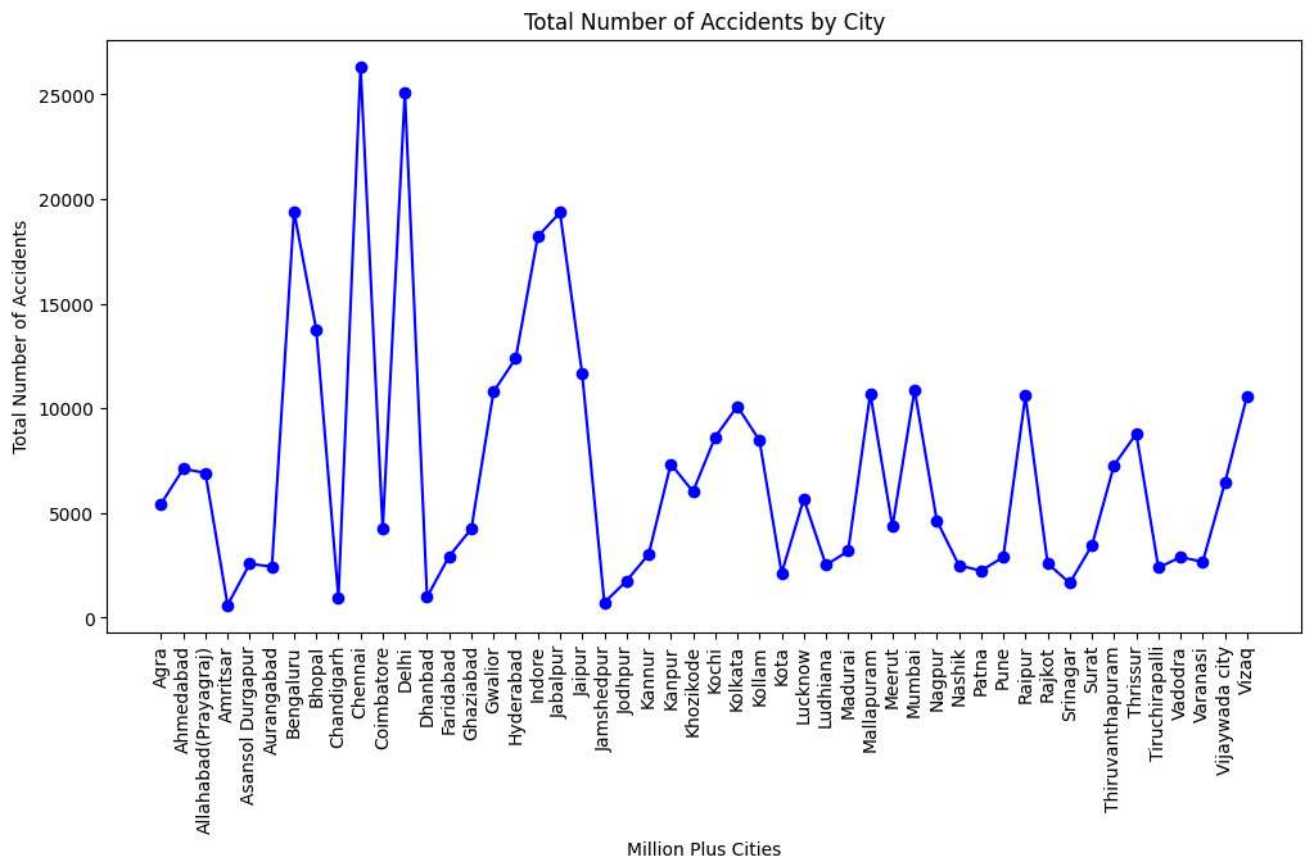


The bubble chart shows the link between the total number of accidents (x-axis) and the total number of injured (y-axis), while the bubble size represents the number of persons killed. Bigger bubbles mean more deaths in that category or city. This makes it easy to see not only how accidents relate to injuries but also where the impact is more severe in terms of fatalities.

Line Chart A line chart is useful for showing trends over time. It connects data points with lines, making it easy to see increases, decreases, or patterns across a timeline or sequence.

A line chart is effective for showing changes in road accidents or injuries across a period. For example, plotting the “Total number of Accidents” by months or years can reveal whether cases are rising, falling, or staying steady. This helps in spotting patterns and understanding the overall trend rather than just looking at single values.

```
1 # Filter only 'Total number of Accidents'
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3
4 # Group by City and sum counts
5 city_trend = accidents.groupby('Million Plus Cities')['Count'].sum()
6
7 # Plot line chart
8 plt.figure(figsize=(12, 6))
9 plt.plot(city_trend.index, city_trend.values, marker='o', linestyle='-', color='blue')
10
11 # Add labels and title
12 plt.title('Total Number of Accidents by City')
13 plt.xlabel('Million Plus Cities')
14 plt.ylabel('Total Number of Accidents')
15
16 # Rotate x-axis labels for readability
17 plt.xticks(rotation=90)
18
19 # Show chart
20 plt.show()
```



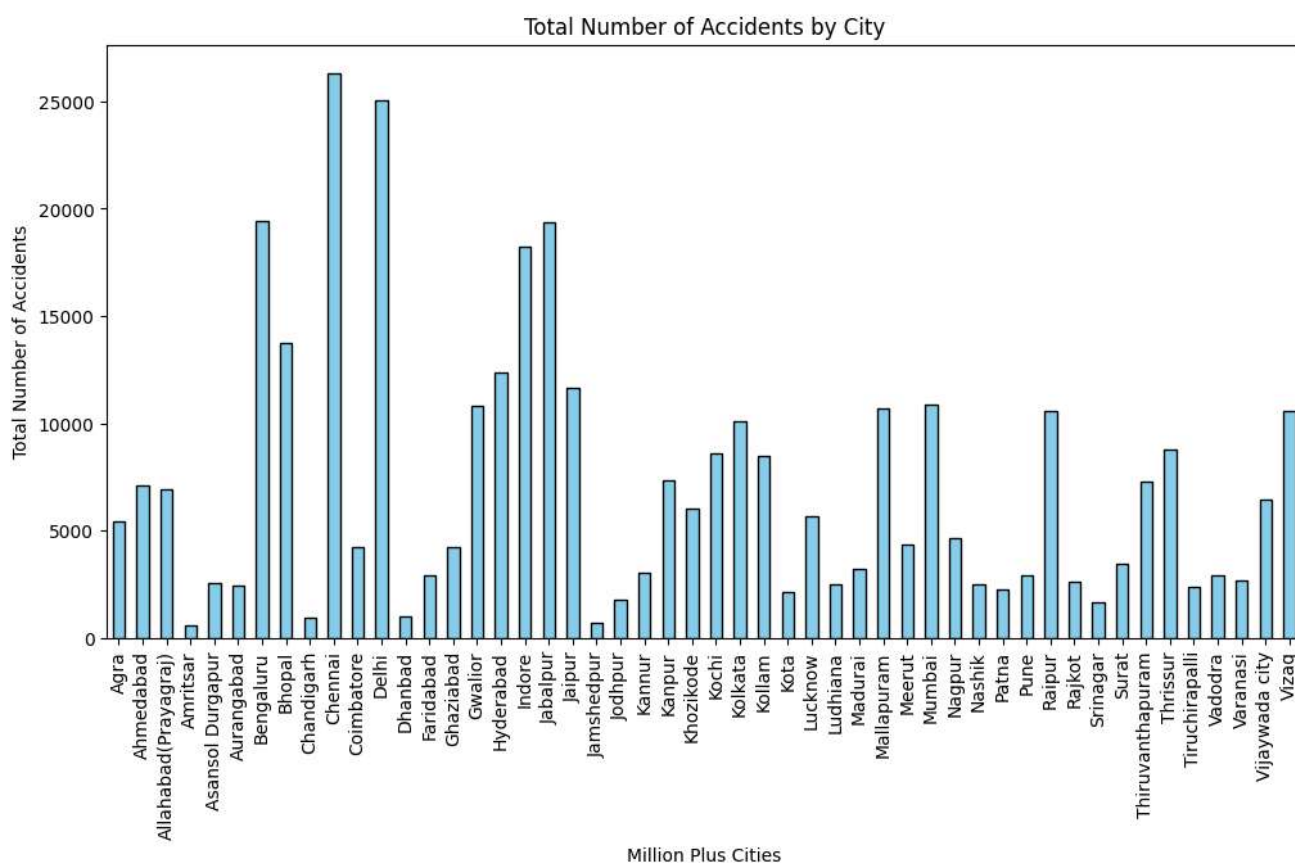
Column Chart A column chart uses vertical bars to compare values across categories. It is effective when you want to highlight differences in size or frequency between groups.

```
1 # Filter only 'Total number of Accidents'
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3
4 # Group by City and sum counts
```

```

5 city_counts = accidents.groupby('Million Plus Cities')['Count'].sum()
6
7 # Plot column chart (vertical bars)
8 plt.figure(figsize=(12, 6))
9 city_counts.plot(kind='bar', color='skyblue', edgecolor='black')
10
11 # Add labels and title
12 plt.title('Total Number of Accidents by City')
13 plt.xlabel('Million Plus Cities')
14 plt.ylabel('Total Number of Accidents')
15
16 # Rotate x-axis labels for readability
17 plt.xticks(rotation=90)
18
19 # Show chart
20 plt.show()

```

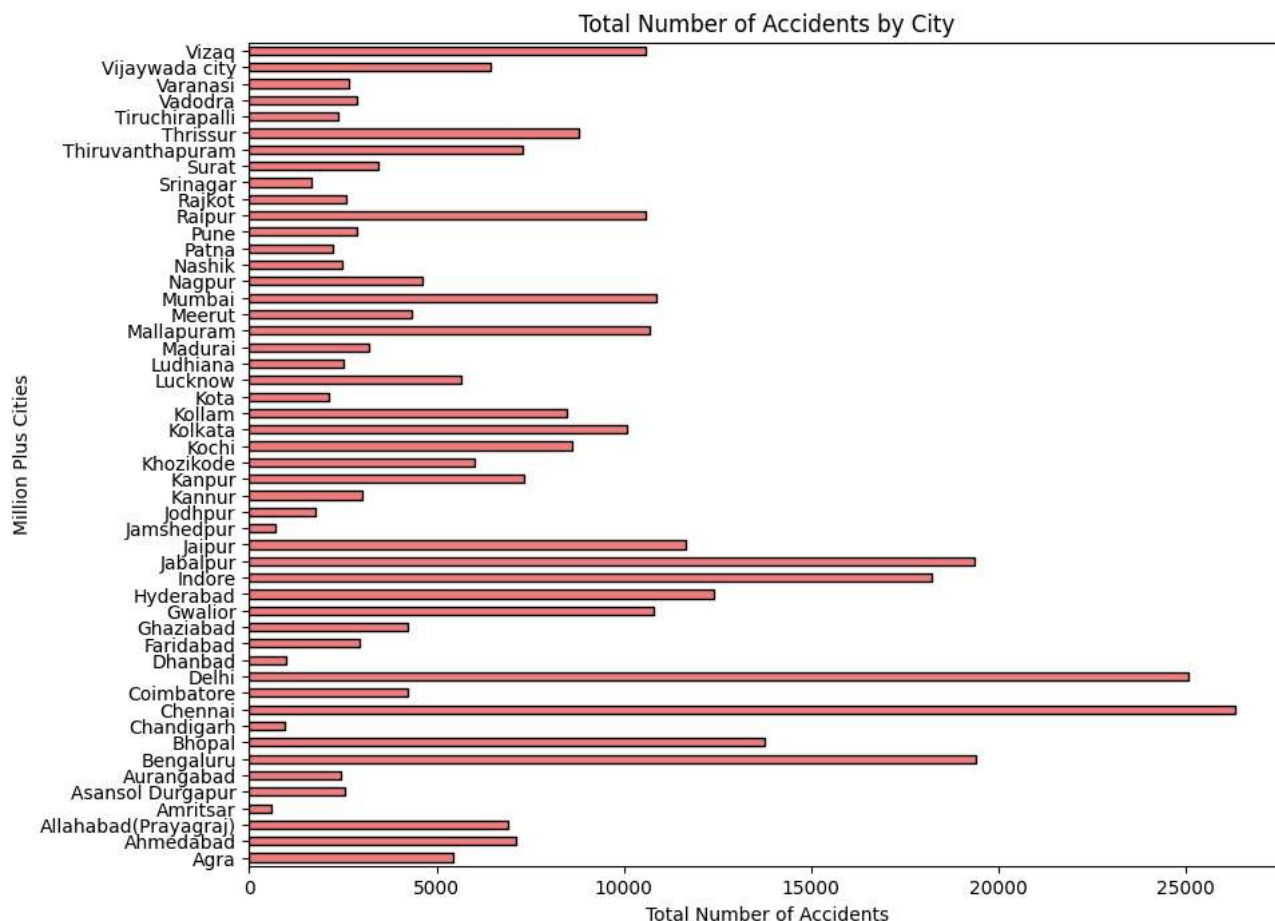


Bar Chart A bar chart uses horizontal bars to compare data. It is especially helpful when category names are long or when you want to easily compare values side by side.

```

1 # Filter only 'Total number of Accidents'
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3
4 # Group by City and sum counts
5 city_counts = accidents.groupby('Million Plus Cities')['Count'].sum()
6
7 # Plot bar chart (horizontal bars)
8 plt.figure(figsize=(10, 8))
9 city_counts.plot(kind='barh', color='lightcoral', edgecolor='black')
10
11 # Add labels and title
12 plt.title('Total Number of Accidents by City')
13 plt.xlabel('Total Number of Accidents')
14 plt.ylabel('Million Plus Cities')
15
16 # Show chart
17 plt.show()

```

**Task 2.**

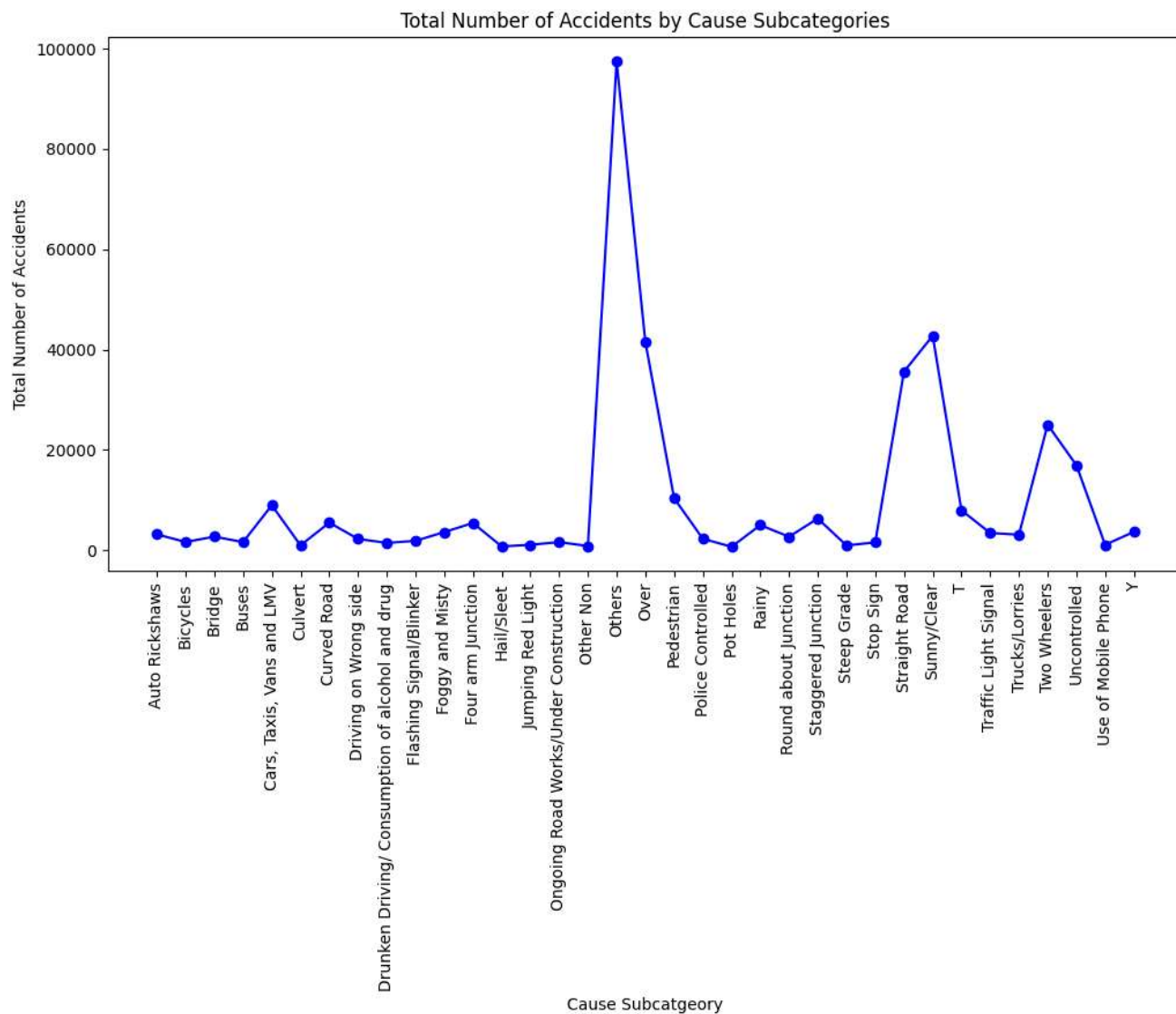
Using the line, column, and bar chart. Visualize the parameter of Cause Subcategory.

LINE

```

1 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
2
3 cs_trend = accidents.groupby('Cause Subcategory')['Count'].sum()
4
5 plt.figure(figsize=(12, 6))
6 plt.plot(cs_trend.index, cs_trend.values, marker='o', linestyle='-', color='blue')
7
8 plt.title('Total Number of Accidents by Cause Subcategories')
9 plt.xlabel('Cause Subcatgeory')
10 plt.ylabel('Total Number of Accidents')
11
12 plt.xticks(rotation=90)
13
14 plt.show()

```

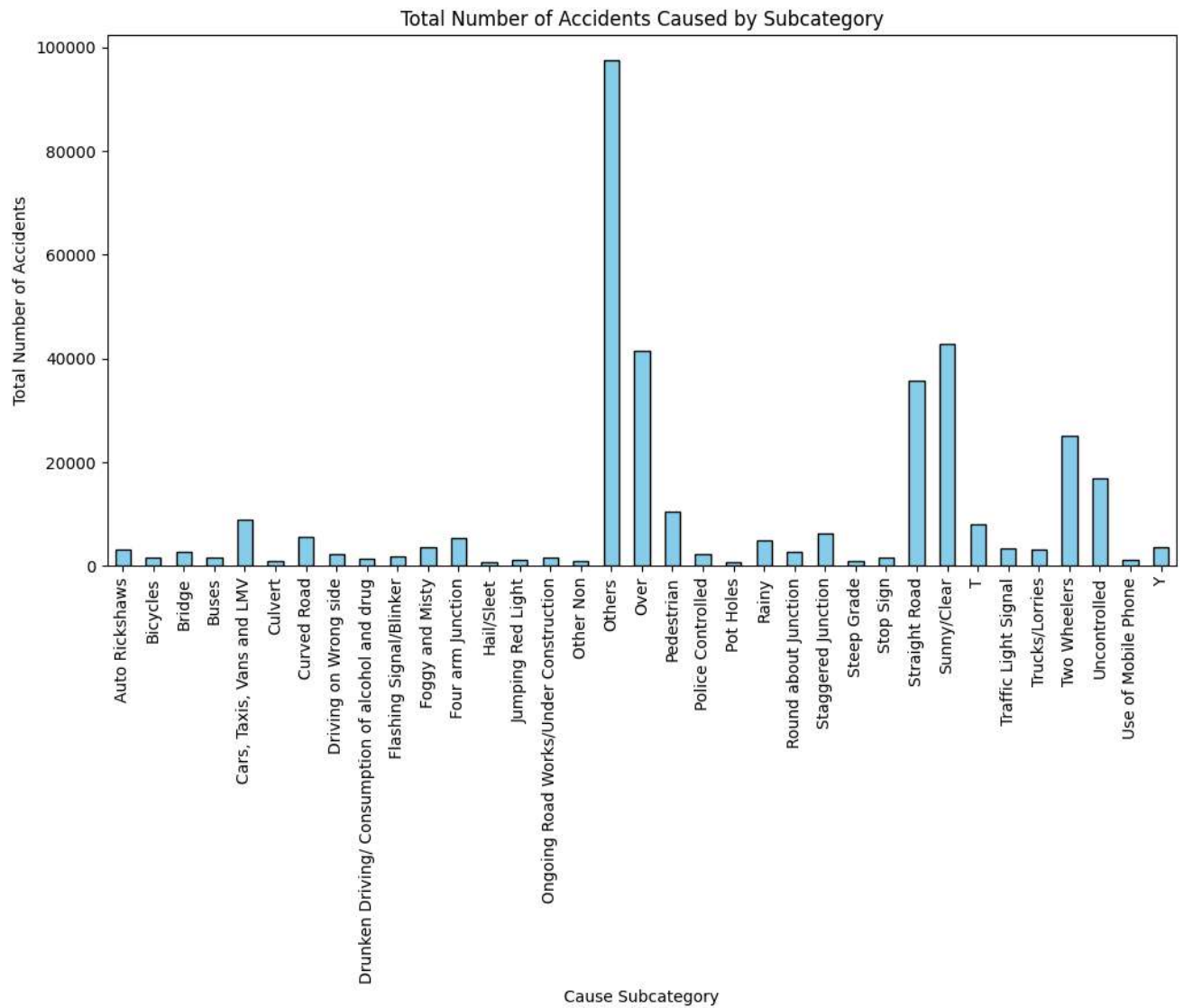


COLUMN

```

1 #Code here
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3
4 subcategory_counts = accidents.groupby('Cause Subcategory')['Count'].sum()
5
6 plt.figure(figsize=(12, 6))
7 subcategory_counts.plot(kind='bar', color='skyblue', edgecolor='black')
8
9
10 plt.title('Total Number of Accidents Caused by Subcategory')
11 plt.xlabel('Cause Subcategory')
12 plt.ylabel('Total Number of Accidents')
13
14 plt.xticks(rotation=90)
15
16 plt.show()

```

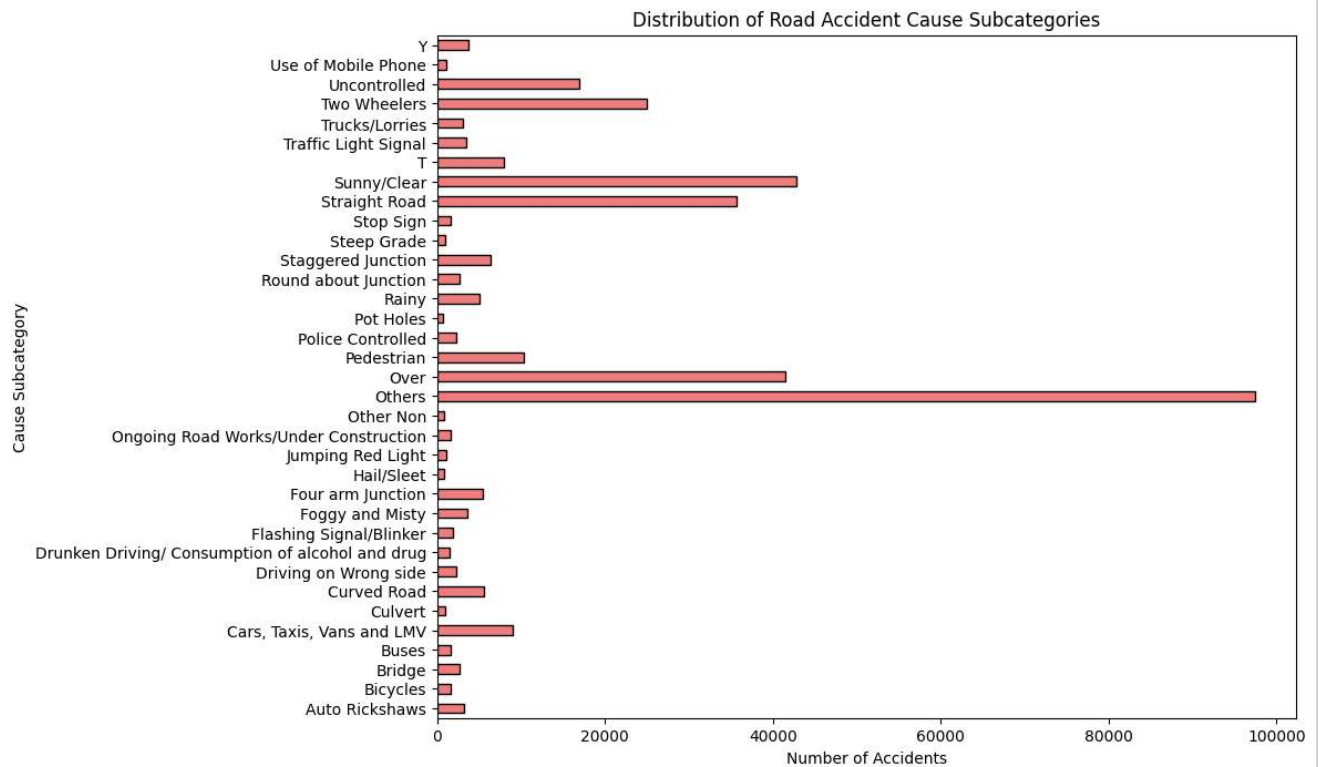


BAR

```

1 #Filter only 'Total number of Accidents'
2 accidents = df[df['Outcome of Incident'] == 'Total number of Accidents']
3
4 # Group by Cause Subcategories and Sum Counts
5 cause_counts = accidents.groupby('Cause Subcategory')['Count'].sum()
6
7 # Plot a bar chart
8 plt.figure(figsize=(10, 8))
9 cause_counts.plot(kind='barh', color='lightcoral', edgecolor='black')
10
11 # Add labels and title
12 plt.title('Distribution of Road Accident Cause Subcategories')
13 plt.xlabel('Number of Accidents')
14 plt.ylabel('Cause Subcategory')
15
16 # Show chart
17 plt.show()

```

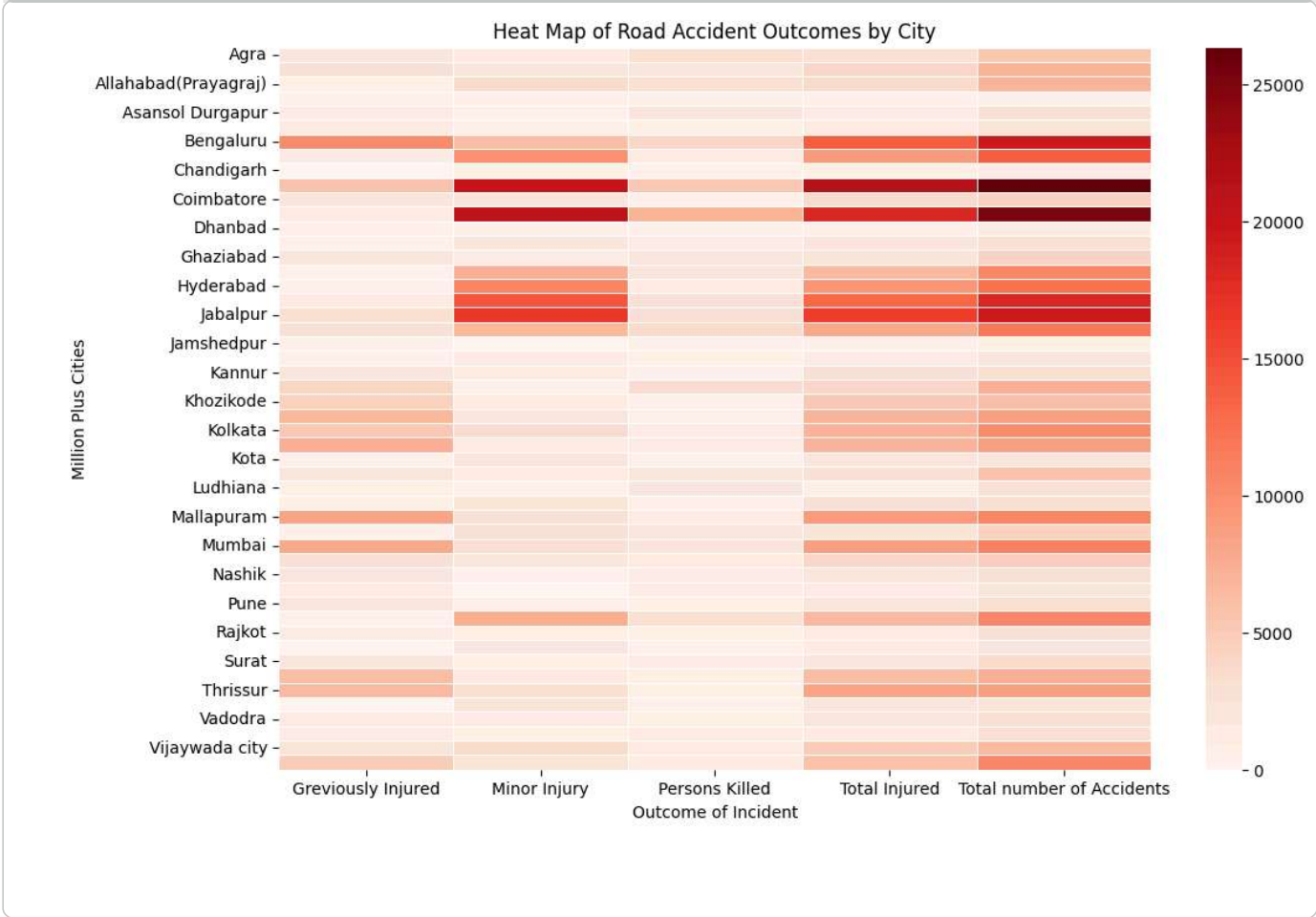


MAPS

Heat Map

```
1 import seaborn as sns
```

```
1 # Pivot table: Cities (rows) vs. Outcomes (columns)
2 heat_data = df.pivot_table(
3     index="Million Plus Cities",
4     columns="Outcome of Incident",
5     values="Count",
6     aggfunc="sum"
7 ).fillna(0)
8
9 # Plot heatmap
10 plt.figure(figsize=(12, 8))
11 sns.heatmap(heat_data, cmap="Reds", linewidths=0.5)
12
13 plt.title("Heat Map of Road Accident Outcomes by City")
14 plt.xlabel("Outcome of Incident")
15 plt.ylabel("Million Plus Cities")
16 plt.show()
```



The heat map shows the intensity of accident outcomes across different cities. Darker colors represent higher numbers, making it easy to see which cities report more accidents, injuries, or fatalities. This type of map is effective for spotting patterns and quickly identifying cities that need more attention in terms of road safety.

Tree Map

The treemap breaks down accidents by cause category and subcategory. Each rectangle’s size represents the number of cases, so larger rectangles mean more frequent causes. This visualization is useful because it highlights which accident causes dominate overall and also shows how subcategories contribute within each main cause.

```
1 !pip install squarify

Collecting squarify
  Downloading squarify-0.4.4-py3-none-any.whl.metadata (600 bytes)
  Downloading squarify-0.4.4-py3-none-any.whl (4.1 kB)
Installing collected packages: squarify
Successfully installed squarify-0.4.4

1 import squarify

1 # Group by Cause category and subcategory
2 treemap_data = df.groupby(["Cause category", "Cause Subcategory"])["Count"].sum().reset_index()
3
4 # Prepare labels
5 labels = treemap_data["Cause category"] + "\n" + treemap_data["Cause Subcategory"] + "\n" + treemap_data["Count"].astype(st
6
7 # Plot treemap with smaller font size
8 plt.figure(figsize=(12, 8))
9 squarify.plot(
10     sizes=treemap_data["Count"],
11     label=labels,
12     alpha=0.7,
13     text_kwargs={'fontsize':8} # correct way to set text size
14 )
```

This treemap visualizes the hierarchical structure of road accident causes and subcategories. The data is organized into three levels of detail, with each node representing a specific category and its associated count. The categories are color-coded and labeled with their respective values.

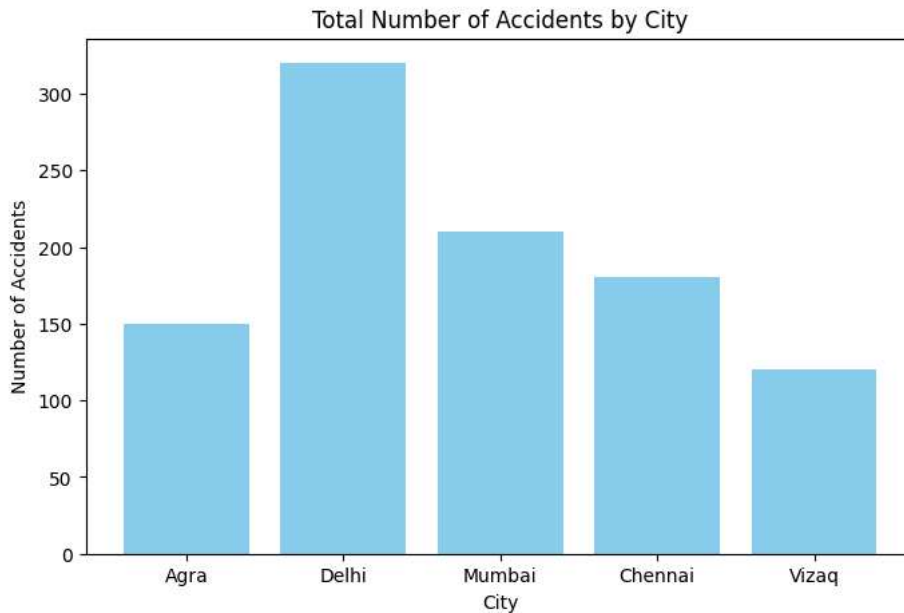
Category	Count
Impacting Vehicle/Object	5589.0
Trucks/Lorries	2938.0
Road Features	8027.5
Other Non	1819.0
Other	7744.0
Impacting Vehicle/Object	22189.0
Other	23555.0
Impacting Vehicle/Object	17263.0
Cars, Taxis, Vans and LMV	16734.75
Other	8075.0
Impacting Vehicle/Object	3470.0
Buses	3472.0
Impacting Vehicle/Object	6691.0
Auto Rickshaws	54356.0
Two Wheelers	15314.0
Four arm Junction	96562.0
Junction	96516.0
Others	96516.0
Traffic Control	6818.0
Police Controlled	2193.0
Weather	15421.0
Rainy	10326.0
Foggy and Misty	18464.0
Weather	126904.0
Sunny/Clear	3228.0
Traffic Violation	32594.0
Others	3228.0
Use of Mobile Phone	123013.0
Traffic Violation	3387.0
Jumping Red Light	4390.0
Traffic Control	9914.0
Stop	50001.0
Traffic Control	6731.0
Uncontrolled	4355.0
Traffic Violation	4355.0
Drunk Driving	4355.0
Consumption of alcohol and drug	5669.0
Traffic Control	5669.0
Flashing Signal/Blinker	104223.0
Road Features	104223.0
Straight Road	104223.0
Junction	96562.0
Others	96562.0

In general, charts can often convey information faster and easier to readers than tables, but in some cases a table is more appropriate. Tables should be used when the:

- ```
1 # Sample data (not in the Road Accidents Dataset)
2 data = {
3 "City": ["Agra", "Delhi", "Mumbai", "Chennai", "Vizaq"],
4 "Total number of Accidents": [150, 320, 210, 180, 120]
5 }
6 df_table = pd.DataFrame(data)
7
8 # Show the table
9 print("Table of Total Number of Accidents by City")
10 print(df_table)
11
12 # Plot the chart
13 plt.figure(figsize=(8, 5))
14 plt.bar(df_table["City"], df_table["Total number of Accidents"], color="skyblue")
15 plt.title("Total Number of Accidents by City")
16 plt.xlabel("City")
17 plt.ylabel("Number of Accidents")
18 plt.show()
```

Table of Total Number of Accidents by City

| City      | Total number of Accidents |
|-----------|---------------------------|
| 0 Agra    | 150                       |
| 1 Delhi   | 320                       |
| 2 Mumbai  | 210                       |
| 3 Chennai | 180                       |
| 4 Vizaq   | 120                       |



### 5. Supplementary Activity:

1. Identify and obtain a dataset in .csv format from reputable sources such as Kaggle or Google Dataset Search.
2. Formulate a clear problem statement that specifies what aspect of the data you aim to analyze or visualize.
3. Apply an appropriate visualization technique by selecting the chart type that best represents the dataset and effectively addresses the stated problem.
4. Provide a comprehensive explanation of the chosen chart, justifying why it is suitable for representing the data and how it contributes to understanding the problem.

## ▼ PROBLEM STATEMENT

*Does the game popularity correlate to positive reviews compared to negative reviews?*

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force\_remount=True).

```
1 df1 = pd.read_csv("/content/drive/MyDrive/CPE 031_Torres/steam_games_dataset.csv")
```

```
1 df1 = df1.drop(columns=['appid', 'developer', 'publisher', 'score_rank', 'userscore', 'owners', 'average_forever', 'average_2weeks'])
2
3 df1.head()
```

|   | name                             | positive | negative |
|---|----------------------------------|----------|----------|
| 0 | Counter-Strike: Global Offensive | 7642084  | 1173003  |
| 1 | Apex Legends                     | 668053   | 326926   |
| 2 | PUBG: BATTLEGROUNDS              | 1520457  | 1037487  |
| 3 | Palworld                         | 358266   | 22443    |
| 4 | Team Fortress 2                  | 1044264  | 117208   |

```
1 df2 = df1.head()
2
3 df2_table = pd.DataFrame(df2)
```